TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN UTN-FRC

TUP

UTN Facultad Regional Córdoba

PROGRAMACIÓN III
BACK END

2025

Spring & Spring Boot

Spring

Spring es un framework para el desarrollo de aplicaciones y contenedor de inversión de control, de código abierto para la plataforma Java

Why Spring?

TUP UTN Facultad Regional Córdoba

Spring hace que programar Java sea más rápido, fácil y seguro para todos. El enfoque de Spring en la velocidad, la simplicidad y la productividad lo ha convertido en el marco Java más popular del mundo.

https://spring.io

Bean

Un **JavaBean** es solo un estándar. Es una clase Java regular, excepto que sigue ciertas convenciones:

1. Todas las propiedades son privadas.
2. Tiene un constructor público sin argumentos.
3. Implementa Serializable.

En Spring, los objetos que forman la columna vertebral de su aplicación y que son administrados por el contenedor Spring IoC se denominan beans. **Un bean es un objeto que es instanciado, ensamblado y administrado por un contenedor Spring IoC**.

Serializable

La serialización de una clase está habilitada por la clase que implementa la interfaz **java.io.Serializable**. Las clases que no implementen esta interfaz no tendrán ninguno de sus estados serializados o deserializados.

Todos los subtipos de una clase serializable son en si serializables.

La interfaz de serialización no tiene métodos o campos y solo sirve para identificar la semántica de ser serializable.

Serialización / Deserialización

TUP UTN Facultad Regional Córdoba

La serialización/deserialización es **un proceso que convierte una instancia en un stream de bytes**. Esos bytes pueden almacenarse en archivos, enviarse a través de una conexión de red, etc. **y tener suficiente información para permitir que una JVM** (al menos, una que conozca el tipo de objeto) **reconstruya el objeto más tarde**, posiblemente en una instancia diferente de la aplicación, ¡o incluso en otra máquina.

Inversion of control (IoC)

TUP UTN Facultad Regional Córdoba

En pocas palabras, la inversión de control (IoC) es **un proceso en el que un objeto define sus dependencias sin crearlas**. Este objeto d**elega el trabajo de construir tales dependencias a un contenedor IoC**.

En Spring Framework, la interfaz **ApplicationContext representa el contenedor IoC**. El contenedor Spring se encarga de **instanciar, configurar y ensamblar objetos conocidos como beans**, así como de gestionar sus ciclos de vida.

Dependency injection (DI)

TUP UTN Facultad Regional Córdoba

En ingeniería de software, la inyección de dependencia **es un patrón de diseño en el que un objeto o función recibe otros objetos o funciones de los que depende**.

La Inyección de Dependencia es un aspecto fundamental del framework Spring, a través del cual el contenedor Spring “inyecta” objetos en otros objetos o “dependencias”.
En pocas palabras, esto permite un acoplamiento flexible de los componentes y transfiere la responsabilidad de administrar los componentes al contenedor.

@Bean

Usamos la anotación @Bean en un método para definir un bean. Si no especificamos un nombre personalizado, el nombre del bean será el nombre del método predeterminado.

@Component

TUP UTN Facultad Regional Córdoba

@Component le permite a Spring detectar nuestros beans personalizados automáticamente. En otras palabras, sin tener que escribir ningún código explícito, Spring:

1. Escanea en busca de clases anotadas con @Component
2. Las instancia y les inyecta las dependencias especificadas.
3. Inyecta estos Beans donde sea necesario

@Component es un “estereotipo” genérico, existen otros más específicos.

Estereotipo

Estereotipo es una clase de “perfil” que define cómo se puede ampliar una metaclase existente como parte de un perfil. Permite el uso de una terminología o notación específica de una plataforma, framework o dominio.

Spring proporciona algunas anotaciones “estereotipadas“ especializadas: @Controller, @Service y @Repository. Todos proporcionan la misma función que @Component.
Todos actúan igual porque todos son anotaciones compuestas con @Component como meta-anotación para cada uno de ellos.

@Bean vs. @Component

@Bean también es una anotación que Spring usa para recopilar beans en tiempo de ejecución, pero no se usa a nivel de clase. En su lugar, anotamos los métodos con @Bean para que Spring pueda almacenar el resultado del método como un bean Spring.

Dentro de nuestra clase anotada con @Configuration, podemos crear un método de generación de beans.

@Controller / @Service / @Repository

TUP UTN Facultad Regional Córdoba

@Controller es una anotación de nivel de clase, que le dice a Spring Framework que esta clase sirve como controlador en Spring MVC.

@Service anota clases en la capa de servicio. La lógica comercial de una aplicación generalmente reside dentro de la capa de servicio, por lo que usaremos la anotación @Service para indicar que una clase pertenece a esa capa.

@Repository anota las clases en la capa de persistencia, que actuará como repositorio de la base de datos. Las clases DAO o Repository generalmente representan la capa de acceso a la base de datos en una aplicación y deben anotarse con @Repository.

@Configuration

TUP UTN Facultad Regional Córdoba

La anotación @Configuration indica que la clase es una fuente de definiciones de beans. También podemos agregarlo a múltiples clases de configuración.

@Autowired

TUP UTN Facultad Regional Córdoba

Autowiring Dependencies: Wiring (cableado) permite que el contenedor Spring **resuelva automáticamente las dependencias entre los beans colaboradores** al inspeccionar los beans que se han definido. Hay cuatro modos de hacer Autowiring:

no: valor predeterminado; esto significa que no se usa autowiring.
byName: el autowiring se realiza en función del nombre de la propiedad, por lo tanto, Spring buscará un bean con el mismo nombre que la propiedad que debe establecerse.
byType: basado en el tipo de propiedad. Si hay más de un bean de ese tipo, el framework lanza una excepción.
constructor: el autowiring se realiza en función de los argumentos del constructor, lo que significa que Spring buscará beans con el mismo tipo que los argumentos del constructor.

@Qualifier

Si hay más de un bean del mismo tipo, podemos usar la anotación @Qualifier para hacer referencia a un bean por nombre.

Initialized Beans

TUP UTN Facultad Regional Córdoba

De forma predeterminada, **el contenedor crea y configura todos los beans singleton durante la inicialización**. Para evitar esto, podemos usar el atributo **lazy-init** con valor verdadero en la configuración del bean.

En consecuencia, el bean solo se inicializará cuando se solicite por primera vez, y no al inicio. **La ventaja de esto es un tiempo de inicialización más rápido**, pero **la contrapartida es que no descubriremos ningún error de configuración hasta que se solicite el bean**, lo que podría ser varias horas o incluso días después de que la aplicación ya se haya estado ejecutando.

TUP UTN Facultad Regional Córdoba

Gracias